

Федеральное государственное автономное образовательное учреждение высшего
образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет информационных технологий

Кафедра «Информатика и информационные технологии»

Направление подготовки/ специальность: 09.03.02 Информационные системы и технологии

ОТЧЕТ

по проектной практике

Студенты: Козырева Елизавета Денисовна, Чиркова Вероника Сергеевна

Группа: 251-333

Место прохождения практики: Московский Политех, кафедра Информационных технологий

Отчет принят с оценкой _____ Дата _____

Руководитель практики: Семёнова Валерия Валерьевна

Москва 2026

ВВЕДЕНИЕ

1. Общая информация о проекте

Название проекта:

Разработка инди-симулятора повседневности «Симулятор раздачи листовок», моделирующего распространенные социальные ситуации и предлагающего игрокам уникальный опыт взаимодействия с городской средой.

Актуальность:

В современной игровой индустрии наблюдается устойчивый рост интереса к «симуляторам повседневности». Несмотря на популярность жанра, на рынке практически отсутствуют проекты, детально моделирующие распространенные социальные ситуации. Разработка данного проекта позволяет заполнить пустующую нишу, предлагая игрокам уникальный опыт взаимодействия с городской средой, что способствует развитию отечественной игровой индустрии в сегменте оригинальных инди-проектов.

Проблематика:

Проект направлен на решение проблемы отсутствия реалистичных и одновременно сатирических игровых моделей, описывающих низкоквалифицированный труд.

Цели и задачи проекта:

Цель: разработка прототипа инди-симулятора «Симулятор раздачи листовок» с базовым геймплеем и пользовательским интерфейсом. Для достижения цели решаются следующие задачи:

1. Анализ предметной области и игровых механик в жанре симуляторов.
2. Формирование концепции, сценария и правил взаимодействия с игровой средой.
3. Проектирование архитектуры и программная реализация механик на игровом движке.
4. Разработка визуального контента и пользовательского интерфейса.
5. Тестирование прототипа и подготовка отчетной документации.

2. Общая характеристика деятельности организации (заказчика проекта)

Наименование заказчика: Роль внутреннего заказчика и эксперта выполняет Центр проектной деятельности Московского Политеха в лице руководителя проекта.

Организационная структура: Учебная группа 251-333, направление подготовки 09.03.02 «Информационные системы и технологии». Команда проекта состояла из двух студентов под руководством ответственного по практике Семеновой Валерии Валерьевны.

Описание деятельности: Заказчик (университет) занимается подготовкой высококвалифицированных IT-специалистов. Проектная практика направлена на приобретение практических навыков командной разработки реального программного продукта, работу с современными API, системами контроля версий и подготовку технической документации.

3. Описание задания по практике

Базовая часть:

1. Настройка Git и репозитория:

- Создать групповой репозиторий на GitHub
- Освоить базовые команды Git: clone, add, commit, push, branch
- Регулярно фиксировать изменения с осмысленными сообщениями

2. Написание документов в Markdown:

- Оформить все материалы проекта в формате Markdown
- Изучить синтаксис Markdown

3. Создание статического веб-сайта:

- Создать сайт на Hugo и опубликовать на GitHub Pages
- Сайт должен включать: Главную, О проекте, Участники, Журнал, Ресурсы
- Оформить страницы графическими материалами

4. Взаимодействие с организацией-партнёром:

- Организовать взаимодействие через куратора проекта
- Написать отчёт в формате Markdown

Вариативная

часть:

Разработка функционального Telegram-бота на языке Python, который реализует компактную текстово-интерактивную версию «Симулятора раздача листовок», позволяющую пользователям в формате мессенджера управлять игровыми ресурсами, проходить ситуативные сценарии и отслеживать статистику.

4. Описание достигнутых результатов по проектной практике

В ходе практики полностью реализован рабочий Telegram-бот «Симулятор раздачи листовок».

Основные реализованные функции:

- Интерактивное управление через адаптивную клавиатуру с набором игровых действий.
- Динамическая генерация ежедневных квестов с учётом погодных и временных модификаторов.
- Трекер статистики с фиксацией успешных действий, отказов и накопленной усталости.
- Анализ текста реакций для точного учёта отказов и провалов.
- Базовые команды: /start (инициализация и показ клавиатуры) и /next_day (сброс игрового дня).
- Надёжная обработка сетевых сбоев и автоматическое восстановление работы при обрыве соединения.

Выполнено:

- Git-репозиторий: https://github.com/Dekim/tgbot_practic
- Подробное руководство по созданию бота (doc/README.md).
- Журнал выполнения работ (doc/TRACKING.md).
- Статический веб-сайт проекта (site/).
- Видео демонстрации работы бота (doc/).

Стек технологий: Python 3, requests, API, Git, Hugo.

Результаты тестирования: Бот успешно работает, корректно отвечает на запросы.

Скриншоты работы сайта

Страница «Главная»



СИМУЛЯТОР РАЗДАЧИ ЛИСТОВОК

Официальный сайт студенческого проекта | Московский Политех



Аннотация проекта

Наш проект представляет собой интерактивную сатирическую инди-игру в жанре социального симулятора повседневности, моделирующую низкоквалифицированный труд промоутера в условиях современной городской среды. Проект сочетает в себе оригинальный гибридный визуальный стиль *Mixed Media* (2D-спрайты персонажей в трехмерном окружении) и глубокую систему нелинейных диалогов, где успех зависит от выстраивания репутации и выбора стиля общения с NPC.

Ключевые особенности MVP:

Движок Unity & C#

Стабильная, оптимизированная кодовая база и полная кроссплатформенность продукта.

Billboard-система

Уникальный визуальный стиль: рендеринг плоских 2D-персонажей в 3D-мире.

Сатирический геймплей

Механики разрушения городской инфраструктуры и динамичные QTE-активности.

Этап 1: Инициация проекта и патентные исследования



Этап 1: Инициация проекта и патентные исследования

Дневник разработки инди-игры Хронологический отчет о выполнении этапов проектной деятельности команды В рамках стартового этапа дисциплины «Проектная деятельность» был...

15 мая 2026 г.

Этап 2: Разработка архитектуры и Mixed Media

Художественный отдел завершил моделирование стартовых локаций "Квартира" в стиле СССР и "Метро". Программисты успешно реализовали на движке Unity базовый скрипт Billboard для...

15 апреля 2026 г.

Этап 3: Финализация MVP и подготовка к защите

На финальном этапе разработки были успешно интегрированы алгоритмы QTE-инвентов для прокачки выносливости промоутера и формулы динамического изменения параметров передвижения...

15 марта 2026 г.



© 2026 Проектная деятельность | Сайт проекта - Powered by Hugo & PaperMod

localhost:1313/posts/post2/

Страница «О проекте»

О проекте

17 мая 2026 г.

Цели и ключевые задачи

Основная цель нашей команды из 26 человек — разработка **высокотехнологичного игрового прототипа** инди-симулятора с проработанным игровым циклом, внутренней экономикой и интерактивным интерфейсом пользователя.

Наша миссия: Создать не просто развлекательный продукт, а ироничное социальное высказывание о ценности человеческого времени и специфике низкоквалифицированного труда в мегаполисе.

Актуальность и уникальность

В геймдев-индустрии стремительно растет спрос на сатирические симуляторы повседневности. Наш продукт успешно выделяется благодаря гибриднему стилю **Mixed Media**, патентной чистоте по ГОСТ 15.011–96 и проработанной кодовой базе на C# в Unity, что полностью защищает проект от юридических рисков при будущей дистрибуции на коммерческих платформах.

Страница «Участники»

Участники проекта

17 мая 2026г.

Функциональная структура команды (26 человек)

Для эффективной координации разработки игрового симулятора команда была разделена на профильные рабочие кластеры:

Рабочий кластер	Участники группы	Зона ответственности
Управление и дизайн	Плешков К., Самодуровский В.	Гейм-дизайн, баланс QTE-активностей
Разработка (Код С#)	Николаенков Д., Плетнева А., Попова Е., Субботина В.	Логика систем взаимодействия в Unity
3D-Художники	Бирюков А., Гагарина Е., Гарченко Д., Кныш С., Смирнова Д.	Моделирование локаций "Квартира" и "Метро"
2D-Арт и Текстуры	Козырева Е., Крутяков М., Лагутенко А., Саяхова Е., Трошестова А., Чиркова В.	Создание спрайтов NPC и элементов UI
Рабочий кластер	Участники группы	Зона ответственности
Управление и дизайн	Плешков К., Самодуровский В.	Гейм-дизайн, баланс QTE-активностей
Разработка (Код С#)	Николаенков Д., Плетнева А., Попова Е., Субботина В.	Логика систем взаимодействия в Unity
3D-Художники	Бирюков А., Гагарина Е., Гарченко Д., Кныш С., Смирнова Д.	Моделирование локаций "Квартира" и "Метро"
2D-Арт и Текстуры	Козырева Е., Крутяков М., Лагутенко А., Саяхова Е., Трошестова А., Чиркова В.	Создание спрайтов NPC и элементов UI
Интерфейс (UI/UX)	Каримбаева А., Кинёв М.	Проектирование радиального меню игры
Сценарий и Звук	Баранов П., Попов Д., Кономанина А.	Нелинейные диалоги, нуарный саунд-дизайн
Аналитика и Маркетинг	Лыхин А., Бородина Е., Власова А., Морев А.	Патентная чистота (ГОСТ), продвижение

Posts

Этап 1: Инициация проекта и патентные исследования

Дневник разработки инди-игры Хронологический отчет о выполнении этапов проектной деятельности команды В рамках стартового этапа дисциплины «Проектная деятельность» был...

15 мая 2026 г.

Этап 2: Разработка архитектуры и Mixed Media

Художественный отдел завершил моделирование стартовых локаций "Квартира" в стиле СССР и "Метро". Программисты успешно реализовали на движке Unity базовый скрипт Billboard для...

15 апреля 2026 г.

Этап 3: Финализация MVP и подготовка к защите

На финальном этапе разработки были успешно интегрированы алгоритмы QTE-ивентов для прокачки выносливости промоутера и формулы динамического изменения параметров передвижения...

15 марта 2026 г.

localhost:1313/posts/post1/

Страница этапов работы над проектом

Этап 1: Инициация проекта и патентные исследования

15 мая 2026 г.

Дневник разработки инди-игры

Хронологический отчет о выполнении этапов проектной деятельности команды

В рамках стартового этапа дисциплины «Проектная деятельность» был проведен детальный анализ рынка инди-симуляторов повседневности. На основе полученных данных сформирована концепция игры «Симулятор раздачи листовок».

Участником команды Лыхиним Артёмом были проведены детальные патентные исследования по ГОСТ 15.011–96, которые полностью подтвердили юридическую чистоту и оригинальность наших программных алгоритмов на C#. Роли среди 26 участников группы успешно распределены, запущен совместный репозиторий на GitHub для контроля версий.

СЛЕДУЮЩАЯ »

Этап 2: Разработка архитектуры и Mixed Media

Проектная деятельность | Сайт проекта

Главная О проекте Участники Журнал Ресурсы

Этап 2: Разработка архитектуры и Mixed Media

15 апреля 2026 г.

Художественный отдел завершил моделирование стартовых локаций "Квартира" в стиле СССР и "Метро".

Программисты успешно реализовали на движке Unity базовый скрипт **Billboard** для корректного отображения двухмерных спрайтов персонажей в трехмерном пространстве (стилистика **Mixed Media**).

« ПРЕДЫДУЩАЯ

Этап 1: Инициация проекта и патентные исследования

СЛЕДУЮЩАЯ »

Этап 3: Финализация MVP и подготовка к защите

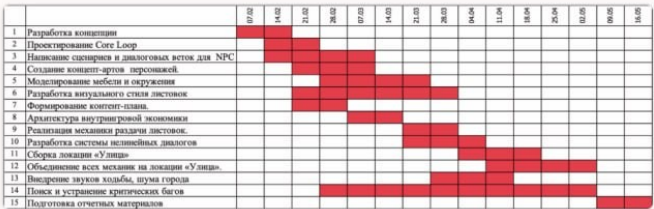
Этап 3: Финализация MVP и подготовка к защите

15 марта 2026 г.

На финальном этапе разработки были успешно интегрированы алгоритмы QTE-ивентов для прокачки выносливости промоутера и формулы динамического изменения параметров передвижения персонажа.

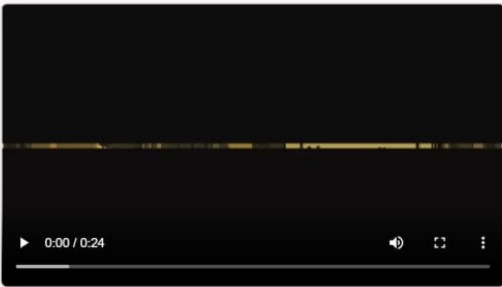
Графические материалы этапа

В рамках планирования и контроля разработки проекта командой была сформирована детальная **диаграмма Ганта (календарный план-график работ)**, отражающая все стадии создания симулятора от концепта до финализации отчетных материалов:



Медиаинформация: Демонстрация геймплея MVP

Ниже представлен видеоотчет с демонстрацией работы игровых механик (Billboard-эффект и радиальное меню взаимодействия) на движке Unity:



Проект успешно апробирован на Дне открытых дверей Московского Политеха. Наша команда полностью подготовила проект к защите!

« ПРЕДЫДУЩАЯ

Этап 2: Разработка архитектуры и Mixed



Полезные ресурсы

17 мая 2026 г.

Источники и материалы проекта

Официальные ресурсы, используемые для аналитики, координации разработки и инженерного обеспечения инди-игры:

Центр проектной деятельности Московского Политеха

[Перейти к регламентам Политеха](#) →

Официальная документация Unity: Спецификация BillboardRenderer

Техническое руководство по рендерингу, использованное для создания визуального эффекта Mixed Media.

[Открыть документацию Unity](#) →

Федеральный фонд стандартов: Регламент ГОСТ 15.011-96

Центр проектной деятельности Московского Политеха

[Перейти к регламентам Политеха](#) →

Официальная документация Unity: Спецификация BillboardRenderer

Техническое руководство по рендерингу, использованное для создания визуального эффекта Mixed Media.

[Открыть документацию Unity](#) →

Федеральный фонд стандартов: Регламент ГОСТ 15.011-96

Государственный стандарт проведения патентных исследований для проверки уникальности игры.

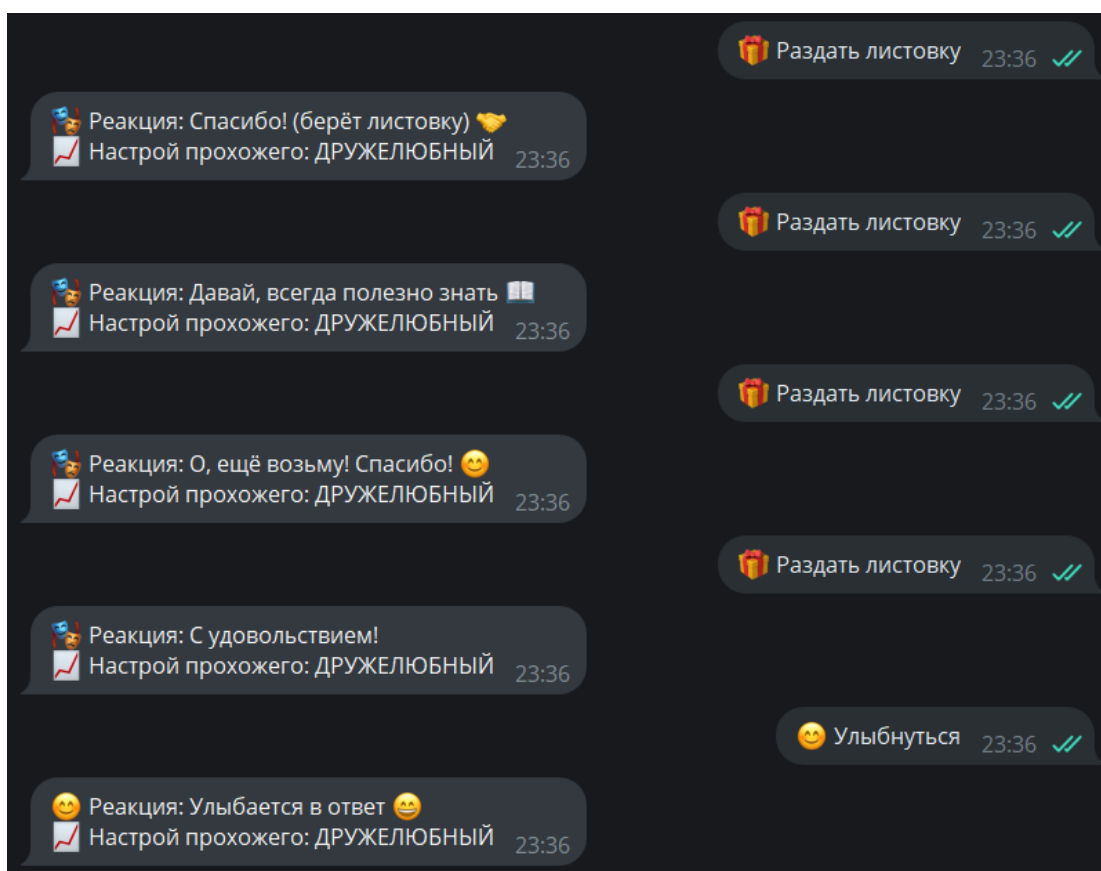
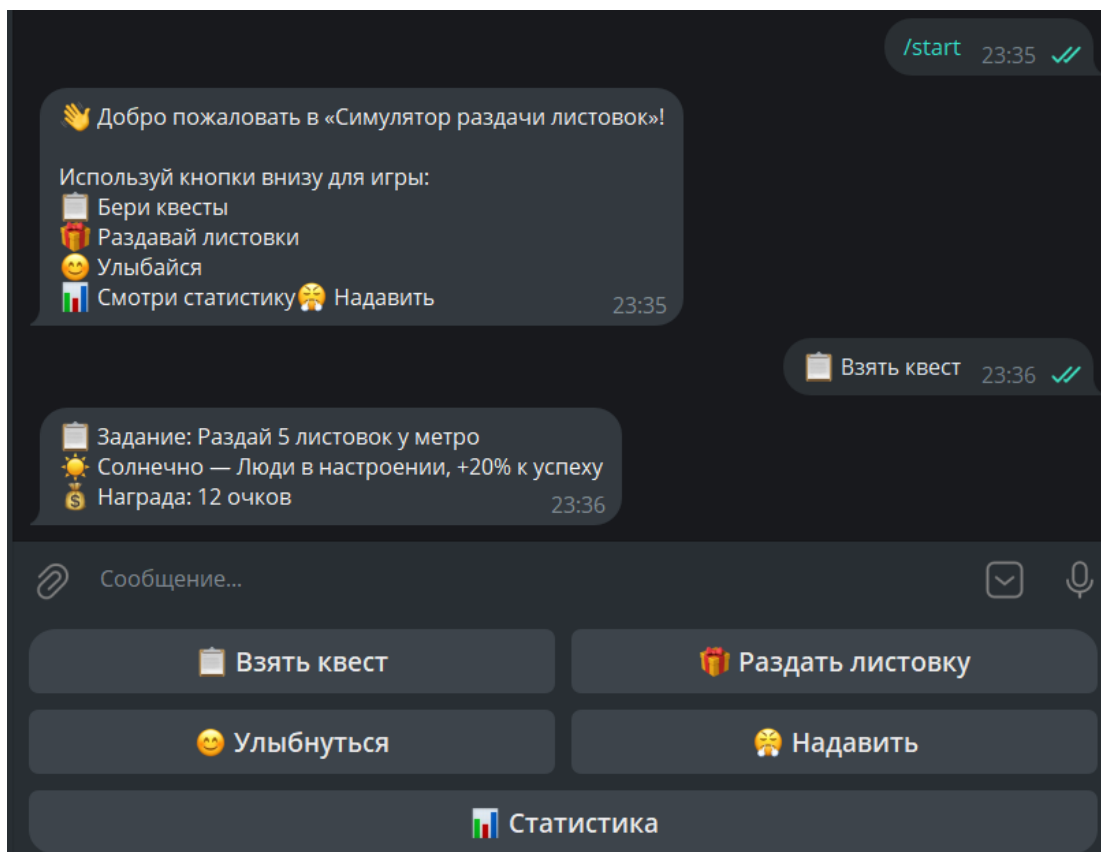
[Проверить реестр ГОСТ](#) →

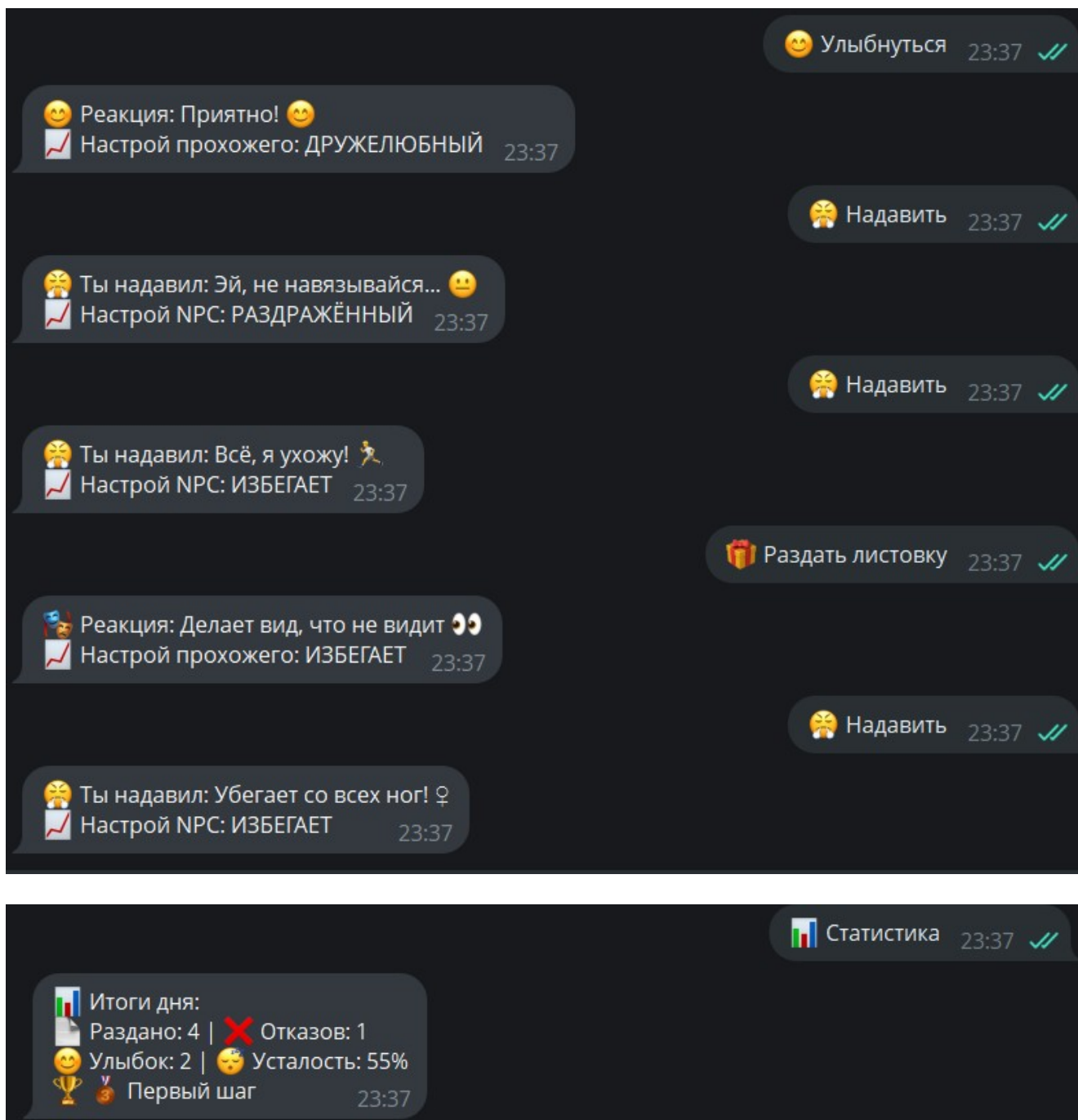
Документация Steamworks: Ценообразование и дистрибуция

Правила платформы Steam Direct по подготовке игрового билда к публикации.

[Открыть Steamworks SDK](#) →

Скриншоты работы бота





Листинг

api.py

```
import requests
from config import BASE_URL

def get_updates(offset, timeout):
    params = {"offset": offset, "timeout": timeout}
    response = requests.get(f"{BASE_URL}/getUpdates", params=params)
    response.raise_for_status()
    return response.json()

def send_message(chat_id, text, reply_markup=None):
```

```

data = {"chat_id": chat_id, "text": text}

if reply_markup:
    import json
    data["reply_markup"] = json.dumps(reply_markup)

response = requests.post(f"{BASE_URL}/sendMessage", json=data)
response.raise_for_status()
return response.json()

```

bot.py

```

from api import get_updates, send_message
from config import POLLING_TIMEOUT
from game.quest_system import generate_daily_quest
from game.npc_fsm import npc_react
from game.stats_tracker import update_stats, get_daily_summary
import time

# Хранилище состояний игроков
players = {}

# Клавиатура с кнопками
MAIN_KEYBOARD = {
    "keyboard": [
        [{"text": "📜 Взять квест"}, {"text": "📄 Раздать листовку"}],
        [{"text": "😊 Улыбнуться"}, {"text": "👉 Надавить"}],
        [{"text": "📊 Статистика"}]
    ],
    "resize_keyboard": True,
    "one_time_keyboard": False
}

def get_or_init_player(chat_id):
    if chat_id not in players:
        players[chat_id] = {
            "npc_state": "НЕЙТРАЛЬНОЕ", # ← Было "NEUTRAL"
            "stats": {"given": 0, "refusals": 0, "smiles": 0, "interactions": 0,
"fatigue": 0, "pushes": 0},
            "quest_active": False
        }
    return players[chat_id]

def handle_message(update):
    if "message" not in update or "text" not in update["message"]:
        return

    chat_id = update["message"]["chat"]["id"]
    text = update["message"]["text"].strip()
    player = get_or_init_player(chat_id)

    # Команда /start с показом клавиатуры
    if text == "/start":
        send_message(
            chat_id,
            "\n Добро пожаловать в «Симулятор раздачи листовок»!\n\n"
            "Используй кнопки внизу для игры:\n"
            "📜 Бери квесты\n"

```

```

        "📄 Раздавай листовки\n"
        "😊 Улыбайся\n"
        "📊 Смотри статистику"
        "👉 Надавить",
        reply_markup=MAIN_KEYBOARD
    )

# Кнопка "Взять квест"
elif text == "📌 Взять квест":
    if not player["quest_active"]:
        player["quest_active"] = True
        quest_text = generate_daily_quest()
        send_message(chat_id, quest_text)
    else:
        send_message(chat_id, "📌 Задание уже активно. Выполни его или начни
новый день через /next_day")

# Кнопка "Раздать листовку"
elif text == "📄 Раздать листовку":
    action = "flyer"
    old_state = player["npc_state"] # Запоминаем старое состояние
    reaction, new_state = npc_react(player["npc_state"], action)
    player["npc_state"] = new_state

    # Считаем успех: если реакция содержит "спасибо" или "возьму" (без учёта
регистра)
    success_words = ["спасибо", "возьму", "давай", "интересно", "ладно",
"удовольствием", "молодец"]
    success = any(word in reaction.lower() for word in success_words)

    update_stats(player["stats"], action, success)
    if not success and action == "flyer":
        update_stats(player["stats"], "refusal", success=False)

    send_message(
        chat_id,
        f"📄 Реакция: {reaction}\n🔧 Настрой прохожего: {new_state}",
        reply_markup=MAIN_KEYBOARD,
    )

# Кнопка "Улыбнуться"
elif text == "😊 Улыбнуться":
    action = "smile"
    reaction, new_state = npc_react(player["npc_state"], action)
    player["npc_state"] = new_state
    success = new_state != "AVOIDING"
    update_stats(player["stats"], action, success)
    send_message(
        chat_id,
        f"😊 Реакция: {reaction}\n🔧 Настрой прохожего: {new_state}"
    )

elif text == "👉 Надавить":
    action = "push"
    reaction, new_state = npc_react(player["npc_state"], action)
    player["npc_state"] = new_state
    success = new_state not in ["AVOIDING", "ANNOYED"]
    update_stats(player["stats"], action, success)
    send_message(
        chat_id,
        f"👉 Ты надавил: {reaction}\n🔧 Настрой NPC: {new_state}",

```

```

        reply_markup=MAIN_KEYBOARD,
    )

    # Кнопка "Статистика"
    elif text == "📊 Статистика":
        summary = get_daily_summary(player["stats"])
        send_message(chat_id, summary)

    # Команда /next_day
    elif text == "/next_day":
        player["quest_active"] = False
        player["npc_state"] = "NEUTRAL"
        send_message(chat_id, "📅 Новый день! Нажми ⚡ Взять квест, чтобы получить задание.")

    # Все остальные сообщения — игнорируем или просим использовать кнопки
    else:
        send_message(
            chat_id,
            "⚠ Пожалуйста, используй кнопки внизу для игры!\n"
            "Если кнопки пропали — напиши /start",
            reply_markup=MAIN_KEYBOARD,
        )

def run_bot():
    last_update_id = 0
    print("📡 Бот запущен. Нажмите Ctrl+C для остановки.")
    while True:
        try:
            updates = get_updates(offset=last_update_id, timeout=POLLING_TIMEOUT)
            if updates.get("ok") and updates.get("result"):
                for update in updates["result"]:
                    last_update_id = update["update_id"] + 1
                    handle_message(update)
        except Exception as e:
            print(f"⚠ Ошибка: {e}")
            time.sleep(1)

```

config.py

```

BOT_TOKEN = "8362304768:AAGs6lTbPhMLrrjPvFeh4ZQnh8soRQ_4l8"
BASE_URL = f"https://api.telegram.org/bot{BOT_TOKEN}"
POLLING_TIMEOUT = 10

```

main.py

```

from bot import run_bot

if __name__ == "__main__":
    run_bot()

```

npc_fsm.py

```

import random

STATES = ["НЕЙТРАЛЬНОЕ", "ДРУЖЕЛЮБНЫЙ", "РАЗДРАЖЁННЫЙ", "ИЗБЕГАЕТ"]

```



```

REACTIONS = {
  "НЕЙТРАЛЬНОЕ": {
    "flyer": [
      "Спасибо! (берёт листовку) ☐",
      "О, интересно... ☐",
      "Ладно, возьму 🤝"
    ],
    "smile": [
      "Кивает в ответ ☐",
      "Улыбается слегка ☐",
      "Приветственно машет ☐"
    ],
    "push": [
      "Хмурится, отходит ☹",
      "Недовольно смотрит ☹",
      "Отстраняется ☐"
    ]
  },
  "ДРУЖЕЛЮБНЫЙ": {
    "flyer": [
      "О, ещё возьму! Спасибо! 😊",
      "Давай, всегда полезно знать ☐",
      "С удовольствием! ",
      "Ты молодец, что раздаёшь! 👍"
    ],
    "smile": [
      "Улыбается в ответ 😊",
      "Смеётся 😄",
      "Поднимает настроение! ☐",
      "Приятно! 😊"
    ],
    "push": [
      "Эй, не навязывайся... ☹",
      "Хватит, я уже взял 😊",
      "Не стоит так стараться ☹"
    ]
  },
  "РАЗДРАЖЁННЫЙ": {
    "flyer": [
      "Хватит уже! ☹",
      "Мне не надо! ☐",
      "Отстань! ☐",
      "Сколько можно?! ☹"
    ],
    "smile": [
      "Игнорирует... ☹",
      "Отворачивается ☐",
      "Не реагирует "
    ],
    "push": [
      "Вызываю охрану! ☐",
      "Всё, я ухожу! ☐",
      "Это harassment! ☹"
    ]
  },
  "ИЗБЕГАЕТ": {
    "flyer": [

```

```

        "Нет! ",
        "Убегают... ",
        "Прячется за дерево ",
        "Делает вид, что не видит "
    ],
    "smile": [
        "Обходит стороной ",
        "Меняет маршрут ",
        "Притворяется, что занят "
    ],
    "push": [
        "Убегает со всех ног! ",
        "Зовёт на помощь! ",
        "Полный игнор! "
    ]
}

def npc_react(current_state, action):
    reactions_list = REACTIONS.get(current_state, {}).get(action, ["Прохожий не понимает действие."])
    text = random.choice(reactions_list)

    if action == "flyer":
        if current_state == "НЕЙТРАЛЬНОЕ":
            new_state = "ДРУЖЕЛЮБНЫЙ" if random.random() > 0.3 else "НЕЙТРАЛЬНОЕ"
        elif current_state == "ДРУЖЕЛЮБНЫЙ":
            new_state = "РАЗДРАЖЁННЫЙ" if random.random() > 0.7 else "ДРУЖЕЛЮБНЫЙ"
        elif current_state == "РАЗДРАЖЁННЫЙ":
            new_state = "ИЗБЕГАЕТ" if random.random() > 0.5 else "РАЗДРАЖЁННЫЙ"
        else:
            new_state = "ИЗБЕГАЕТ"
    elif action == "smile":
        if current_state in ["НЕЙТРАЛЬНОЕ", "ДРУЖЕЛЮБНЫЙ"]:
            new_state = "ДРУЖЕЛЮБНЫЙ"
        elif current_state == "РАЗДРАЖЁННЫЙ":
            new_state = "НЕЙТРАЛЬНОЕ" if random.random() > 0.6 else "РАЗДРАЖЁННЫЙ"
        else:
            new_state = "ИЗБЕГАЕТ"
    elif action == "push":
        if current_state == "НЕЙТРАЛЬНОЕ":
            new_state = "РАЗДРАЖЁННЫЙ"
        elif current_state == "ДРУЖЕЛЮБНЫЙ":
            new_state = "РАЗДРАЖЁННЫЙ"
        elif current_state == "РАЗДРАЖЁННЫЙ":
            new_state = "ИЗБЕГАЕТ"
        else:
            new_state = "ИЗБЕГАЕТ"
    else:
        new_state = current_state

    return text, new_state

```

quest_system.py

```

import random

QUESTS = [
    {"text": "Раздай 5 листовок у метро", "base_reward": 10},
    {"text": "Улыбнись каждому 3-му прохожему", "base_reward": 5},

```

```

        {"text": "Избегай полиции в парке", "base_reward": 15},
        {"text": "Поговори с незнакомцем о погоде", "base_reward": 8}
    ]

MODIFIERS = [
    {"condition": "☁ Дождь", "text": "Листовки мокнут, эффективность -30%",
    "mult": 0.7},
    {"condition": "☀ Солнечно", "text": "Люди в настроении, +20% к успеху",
    "mult": 1.2},
    {"condition": "🌃 Вечер пятницы", "text": "Все спешат, но открыты к общению",
    "mult": 1.0}
]

def generate_daily_quest():
    quest = random.choice(QUESTS)
    mod = random.choice(MODIFIERS)
    reward = int(quest["base_reward"] * mod["mult"])
    return f"📜 Задание: {quest['text']}\n{mod['condition']} — {mod['text']}\n💰 Награда: {reward} очков"

```

stats_tracker.py

```

# Условия достижений
ACHIEVEMENTS = {
    "first_step": {"cond": lambda s: s["given"] >= 1, "title": "👣 Первый шаг"},
    "smile_master": {"cond": lambda s: s["smiles"] >= 5, "title": "😊 Мастер улыбки"},
    "relentless": {"cond": lambda s: s["given"] >= 20, "title": "📦 Неутомимый курьер"},
    "survivor": {"cond": lambda s: s["interactions"] >= 30 and s["fatigue"] > 80,
    "title": "👤 Выживший"}
}

def update_stats(stats, action, success=True):
    if action == "flyer" and success: stats["given"] += 1
    if action == "refusal": stats["refusals"] += 1
    if action == "smile": stats["smiles"] += 1
    if action == "push":
        stats["pushes"] = stats.get("pushes", 0) + 1 # Счётчик надавливаний
        stats["interactions"] += 1
        stats["fatigue"] = min(100, stats["fatigue"] + 5)
    return stats

def get_daily_summary(stats):
    """Генерирует итоговую сводку"""
    unlocked = [a["title"] for a in ACHIEVEMENTS.values() if a["cond"](stats)]
    ach_text = "\n🏆 " + ", ".join(unlocked) if unlocked else "\n🏆 Достижений пока нет"

    return (f"📊 Итоги дня:\n"
            f"📄 Раздано: {stats['given']} | 🚫 Отказов: {stats['refusals']}\n"
            f"😊 Улыбок: {stats['smiles']} | 😴 Усталость: {stats['fatigue']}%{ach_text}")

```

ЗАКЛЮЧЕНИЕ

В ходе проектной практики студенты успешно выполнили все задачи базовой и вариативной частей.

В рамках базовой части были освоены навыки работы с Git/GitHub, создания статических сайтов на Hugo, написания технической документации в формате Markdown, а также организовано взаимодействие с организацией-партнёром через куратора проекта.

В рамках проектной практики команда успешно разработала функциональный Telegram-бот на языке Python, реализующий компактную текстово-интерактивную версию «Симулятора раздачи листовок». Все поставленные цели и задачи были выполнены в полном объёме.

Выводы о проделанной работе:

- Закреплены и углублены практические навыки программирования на Python.
- Освоены современные инструменты командной разработки (Git, GitHub).
- Приобретён опыт создания технической документации и презентационных материалов (веб-сайт).
- Развиты навыки командной работы, распределения задач и тайм-менеджмента.

Ценность выполненных задач для заказчика: Созданный Telegram-бот наглядно подтверждает освоение командой современных технологий разработки мессенджер-решений и веб-сервисов. Проект выполнен в строгом соответствии с регламентом учебной практики и вносит ощутимый вклад в укрепление цифровых компетенций и IT-инфраструктуры университета.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Официальная документация Git — <https://git-scm.com/book/ru/v2>
2. Документация Hugo — <https://gohugo.io/>
3. Руководство по синтаксису Markdown — <https://www.markdownguide.org>

ПРИЛОЖЕНИЯ:

- Ссылка на репозиторий: <https://github.com/Beatris220/my-project>
- Ссылка на сайт: <https://beatris220.github.io/my-project/>
- Ссылка на tg-бота: @Steamleaf_simulator_bot